

CampusHub 类图

团队： 暴风星云裂 | **项目：** CampusHub | **日期：** 2026年5月16日 **阶段：** P3 详细设计
范围： 核心业务类图

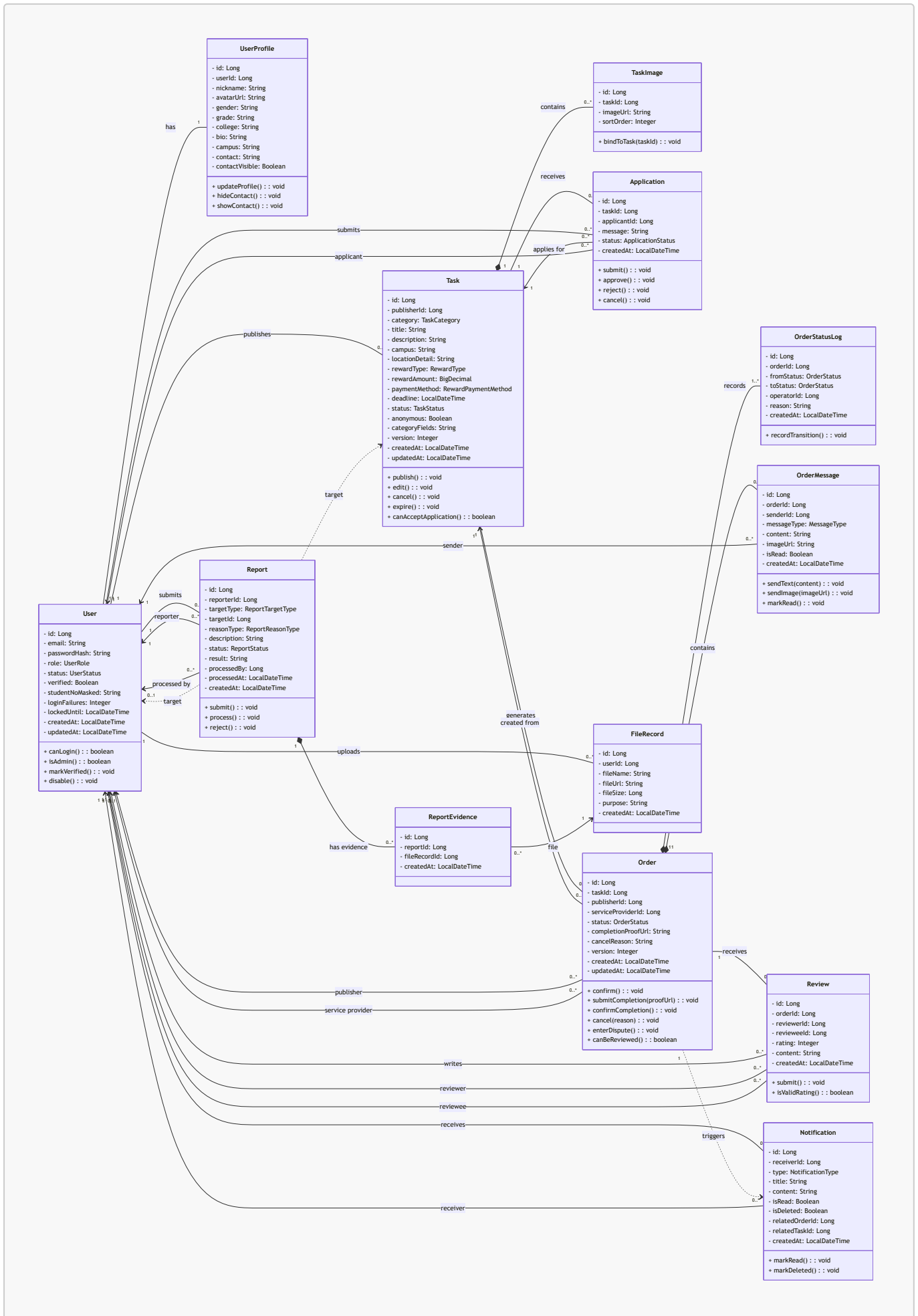
1. 设计范围说明

本类图围绕 CampusHub 的核心业务闭环展开：

用户注册登录 -> 发布需求 -> 接单申请 -> 形成订单 -> 订单内交流 -> 完成订单 -> 提交评价 -> 发起举报

本图优先覆盖 P1/P2 已确定的核心模块，不在第一版类图中展开验证码、收藏、公告、申诉、处罚、黑名单等扩展对象，以保证结构清晰、重点突出。

2. 核心类图



3. 关系说明

- **User** 是平台参与者核心，负责发布需求、申请接单、发送消息、提交评价与举报。
- **Task** 是需求发布对象，**Application** 表示接单申请，申请被确认后生成 **Order**。
- **Order** 是执行主线中心对象，状态变化写入 **OrderStatusLog**，沟通消息写入 **OrderMessage**。
- **Notification** 与 **OrderMessage** 不同：前者是系统通知，后者是订单内聊天消息。
- **Review** 表示订单完成后的双向评价，**Report** 当前实现覆盖任务和用户举报；枚举中保留了消息、评价等后续扩展目标。

4. 第一版取舍说明

本版类图优先突出主业务骨架，因此未纳入以下扩展类：

- **VerificationCode**
- **Favorite**
- **CreditLog**
- **Appeal**
- **Punishment**
- **Announcement**
- **FileRecord**
- **BlacklistRelation**

这些对象可在后续子类图或扩展类图中补充。

5. 设计模式应用

5.1 策略模式

适用场景： 不同需求类型的字段校验与业务规则处理

CampusHub 的 **Task** 采用统一实体建模，但不同类别需求的校验规则并不相同。例如：

- 快递代取需要校验快递公司、取件地点、取件码、送达地点
- 二手交易需要校验商品分类、新旧程度、售价、商品图片
- 失物招领需要校验物品名称、地点、时间和特征描述

因此，在我们的后续实现中，适合为不同任务类别设计独立的校验策略，例如：

- **ExpressTaskValidator**
- **SecondHandTaskValidator**
- **LostFoundTaskValidator**

由统一的任务发布服务根据 **TaskCategory** 选择对应策略执行校验。

为什么在这里使用这个模式？

- 不同类别需求有明确差异化规则
- 这些规则天然适合拆分成独立处理逻辑
- 新增需求类别时，只需新增一个策略类，较少影响既有代码

如果不用这个模式会怎样？

最常见的结果是把所有类别逻辑都堆进一个大方法里，例如：

- `if category == 快递代取`
- `else if category == 二手交易`
- `else if category == 失物招领`

这样在首版代码量小时还能勉强维护，但随着类别增加，发布/编辑需求的逻辑会迅速膨胀，后续新增类别时也必须频繁修改旧代码，扩展性和可读性都会变差。

5.2 工厂模式

适用场景： 按通知类型创建系统通知对象

在我们的 CampusHub 系统中，存在多种系统通知，例如：

- 接单申请通知
- 订单状态变更通知
- 评价通知
- 举报处理结果通知

这些通知虽然都属于 `Notification`，但标题、内容模板、接收人和触发来源并不相同。因此，在我们的后续实现中，适合通过工厂模式统一通知对象创建过程，例如：

- `NotificationFactory`
- `ApplicationNotificationFactory`
- `OrderStatusNotificationFactory`

或者由一个统一工厂方法根据通知类型构造不同通知内容。

为什么在这里使用这个模式？

- 通知对象创建规则集中，便于统一管理
- 可以避免业务服务层到处手写 `new Notification(...)`
- 后续新增通知类型时，扩展点更清晰

如果不用这个模式会怎样？

通常会出现这样的情况：

- `OrderService` 里自己拼通知内容
- `ReviewService` 里自己拼通知内容
- `ReportService` 里又自己拼一套通知内容

结果是通知创建逻辑分散在多个服务中，格式容易不统一，后期修改通知文案或新增字段时，也要在多个地方重复改动。

5.3 本项目中的取舍说明

本阶段只选择两种最贴合当前系统、也最容易与后续 Java 实现对应的设计模式：

- 策略模式：解决任务类别差异化处理问题
- 工厂模式：解决通知对象创建分散问题